Analyst Methods Handbook

Threat Hunt / DFIR Methodology Reference

Collection Plan Methodology

Purpose. Walk onto a network, look at how it is built and (when you have it) who is coming after it, and decide *what to collect, where to collect it, how, and why* — before you drown in data you can't use or miss the data you needed. This is the front of the pipeline:

```
  [ COLLECT ]  ->  [ BASELINE ]  ->  [ VISUALIZE ]  ->  [ DETECT ]
collection plan   what's normal     Kibana dashes     Suricata/Zeek/Defend/KQL
```

Everything downstream inherits the decisions you make here. A dashboard can only show what you collected; a baseline can only normalize what you collected; a detection can only fire on what you collected. Collection is the first and most expensive mistake to get wrong.

---

1. The organizing idea: collect at Terrain x Threat

The instinct on a new network is to "collect everything." You can't, and you shouldn't — you have finite sensors, finite ingest, finite analysts, and some collection is actively harmful (crashing an OT device, tipping off an actor you're watching). So you need a principle that tells you *where the value is*.

The value is at the **intersection of terrain and threat:**

- **Terrain** = what exists and what matters. The network build, the segments, the choke points, the crown jewels, the data flows. Terrain tells you *where things can happen and where it would hurt.*
- **Threat** = what the adversary does. Their TTPs, expressed as ATT&CK techniques, each of which leaves evidence in a specific place and data source. Threat tells you *what to look for and what records it leaves.*

```
               THREAT (what the adversary does)
               techniques -> required telemetry
                        |
                        v
TERRAIN  ------->  [ CROSSWALK ]  ------->  COLLECTION REQUIREMENTS
where things       where a technique        what / where / how /
happen & hurt      would land on YOUR        why / priority / owner
                   terrain, and what
                   data source sees it
```

Where a technique the threat uses would land on terrain you care about, and a data source exists that would see it — that is a collection requirement. Rank those requirements, place sensors and request logs to satisfy the top ones within your constraints, then prove your coverage back against both axes.

Two framings feed the crosswalk and you use both:

- **Threat-informed** — start from the actor. "APTxx uses T1021.002 (SMB/admin shares) for lateral movement" → you need SMB session + auth telemetry on the segments where lateral movement would occur. Precise, but only as good as your intel.
- **Coverage-informed** — start from the platform. "This is a Windows AD environment" → there is a known default set of techniques and a known default telemetry set that covers most of them (process creation, auth, PowerShell, persistence, network). Broad, works with zero intel, and catches the actor your intel didn't name.

Rule of thumb: coverage-informed sets your floor (you always collect the default visibility set for the platform); threat-informed raises specific areas above the floor (you collect *extra* where a named actor concentrates). If you have no intel, you still have a complete plan from the coverage side alone.

---

2. Inputs — what you gather before you plan

You can't crosswalk terrain against threat until you've modeled both. Collect these first:

Network build / terrain

- Network diagram(s) — L2/L3, VLANs/segments, routing, egress points, DMZ, management network, OT/ICS boundary if any.
- Asset inventory — hosts, roles (DC, file server, web, DB, workstation, jump host), OS/version, owners, criticality.
- Data-flow / service map — who talks to whom, on what ports, in which direction; where the crown-jewel data lives and moves.
- Choke points — where traffic concentrates (egress, core/distribution, inter-segment routers, VPN concentrators).
- Existing visibility — what's already logging, what's already forwarding, where sensors already sit. Don't re-collect what you already have.

Threat intel (when available)

- Named actor(s) and their TTPs → translate to ATT&CK technique IDs.

- Known IOCs (hashes, domains, IPs, JA3/JA4) — these feed detection (intel framework / indicator matching), not collection placement, but note them.
- Targeted sectors/data — tells you which crown jewels are in play.
- If **no intel**: use the platform default technique set (coverage-informed). Don't stall waiting for intel.

Engagement / constraints

- Objective — hunt known compromise? establish monitoring? validate after an incident? This changes priority weighting.
- ROE / authorities — what you are allowed to touch, deploy, decrypt, and collect. Hard boundary.
- Environment — **range** vs **real client** (drives your assumption markers, Section 7).
- Resources — sensor count (often **one**), ingest budget, analyst hours, change-control friction.

---

3. The method, step by step

```
1. MAP TERRAIN      enumerate assets, segments, choke points,
                    crown jewels, flows -> terrain model

2. CHARACTERIZE     intel -> ATT&CK techniques -> required data sources
   THREAT           (no intel? platform default technique set)

3. CROSSWALK        overlay each technique on terrain: where would it
                    manifest here, what data source would see it?
                    -> candidate collection requirements

4. PRIORITIZE       score each = likelihood x detection-value x feasibility
                    -> ranked requirement list

5. PLACE / REQUEST  network: choose sensor location(s)  (Section 5)
                    host:    request logs / deploy agents  (Section 6)

6. DOCUMENT         write each as a collection-requirement record
                    (Section 8) -> the collection plan

7. VALIDATE         map planned collection back onto ATT&CK + terrain;
                    find and log the gaps (Section 10)
```

Step 2 detail — technique to data source. Each ATT&CK technique has associated *data sources / data components* (ATT&CK publishes these). That mapping is the bridge from "what the actor does" to "what I must collect." Examples:

Technique (example)	Primary data source(s)	Where it lands
T1059.001 PowerShell	Script block logging (4104), process creation (Sysmon 1 / 4688), module logging	Endpoint
T1021.002 SMB/Admin shares	Logon (4624 type 3), network share (5140/5145), SMB flows	Endpoint + network
T1071 App-layer C2	Network connection, DNS, TLS metadata, proxy/flow	Network sensor
T1053.005 Scheduled task	Task events (4698/4702), Sysmon 1, registry	Endpoint
T1003 Cred dumping (LSASS)	Process access (Sysmon 10), EDR behavioral, memory	Endpoint (EDR)
T1048 Exfil over alt proto	Flow records, bytes-out, DNS, protocol metadata	Network sensor

The point of the table is the *pattern*: some techniques are only visible on the host, some only on the wire, many on both — and that tells you whether a requirement is a sensor-placement decision or a log-request/agent decision.

Step 4 detail — the priority score. Rank every candidate requirement by three factors (1-5 each; multiply, or weight if you prefer):

- **Threat likelihood** — how probable is this technique on this terrain, given intel + platform? (No intel → rate by prevalence on the platform.)
- **Detection value** — if you collect this, how much does it actually buy you? High for telemetry that covers many techniques (process creation, auth, DNS); low for narrow one-off sources.
- **Feasibility** — can you realistically collect it? Accounts for sensor access, ingest cost, change control, host fragility, ROE. A perfect requirement you can't satisfy scores zero in practice.

```
priority = likelihood x detection_value x feasibility
```

This forces the hard trade honestly: a high-value source you cannot feasibly collect drops below a medium-value source you can. You collect the plan you can actually execute, not the plan you wish you had.

---

4. Host + network are one plan, not two

Keep them in one document because the same technique often shows on both, and the two collection surfaces *cover each other's blind spots:*

- The **network sensor** sees breadth cheaply (every host that crosses the sensor, no agent needed) but goes blind on encryption, on same-segment host-local activity, and on anything that never crosses the sensor.
- **Host collection** sees depth (process lineage, script content, in-memory behavior, local file writes) but only on hosts you've instrumented, and it's expensive per host.

So the plan reads: *network sensor for breadth and the wire; host collection to add depth where the terrain is valuable or the network sensor is blind.* The two sections below are halves of the same decision.

---

5. Network collection — sensor placement

The one-sensor problem

Assume you get **one** sensor. Where it sits decides what the entire network side of your operation can see, so this is the highest-leverage single decision in the plan. The question is which axis of traffic you buy visibility into:

```
       NORTH  (Internet / external)
         ^
         |   <-- egress / boundary choke:
         |       sees C2 beacon-out, ingress, exfil,
  [ PERIMETER / EGRESS ]        inbound exploit
         |
  ======================  <-- core / distribution choke:
         |                    sees east-west, lateral movement,
  [ CORE / DISTRIBUTION ]      internal recon, crown-jewel access
         |
  [ ACCESS / HOSTS ]
         v
       SOUTH  (internal)
```

North-south (egress/boundary) catches the adversary's *conversations with the outside world* — C2 check-ins, data exfiltration, inbound exploitation, downloads. If the engagement is an external threat actor, C2, or exfil, this is usually the default. It is also the cheapest high-value spot because nearly all external-facing badness funnels through it.

East-west (core/distribution) catches *movement inside the perimeter* — lateral movement, internal recon, credential reuse, access to crown jewels. If the engagement is an insider, an already-established foothold, or protecting specific internal assets, this axis wins.

Picking the one spot — decision logic

```
Start.
  |
  Q1: Is the engagement external threat / C2 / exfil / unknown?
  |      YES -> default to the EGRESS / boundary choke point.
  |      NO  -> continue.
  |
  Q2: Is the engagement internal (insider, established foothold,
  |   protect specific crown jewels)?
  |      YES -> default to the CORE / distribution choke that
  |             carries the crown-jewel flows.
  |      NO  -> continue.
  |
  Q3: Is there ONE choke point that carries both the egress AND
  |   the bulk of east-west? (small/flat networks often have this)
  |      YES -> put it there; you get both axes at once.
  |      NO  -> fall back to egress (broadest default coverage).
  |
  Then adjust for feasibility (below) before committing.
```

Scoring candidate locations

Rate each physical candidate and let feasibility veto:

- **Coverage breadth** — how many hosts/flows of interest cross this point.
- **Choke centrality** — does traffic concentrate here, or will you miss flows that route around it?
- **Crown-jewel visibility** — does the sensor see the traffic to/from what matters most?

- **Decryption availability** — can you see inside TLS here (proxy break, keys), or only metadata? Placement near a decryption point is worth a lot.
- **Feasibility** — SPAN capacity on the switch, physical TAP access, throughput/oversubscription, asymmetric routing (do you see both directions?), and whether you're allowed to tap at all.

The secondary sensor

If scope allows a second sensor, **do not duplicate the first — cover the first one's blind spot.** The whole value of #2 is complementary coverage:

- If #1 is at egress (north-south), put #2 at the core/distribution for east-west, or on the segment holding the crown jewels.
- If #1 is at the core, put #2 at egress to pick up C2/exfil.
- Special case: if there's an OT/ICS boundary or a high-value enclave, #2 on that boundary is often worth more than generic east-west — it's terrain you can't afford to be blind to and usually can't instrument at the host.

Practical wire notes (carry these on-site)

- **TAP vs SPAN** — TAP is passive, full-fidelity, no drops, needs physical access + the right media. SPAN is config-only but drops under load, can't exceed the port's bandwidth, and may miss errors. Prefer TAP for a monitored choke; SPAN when you can't physically tap.
- **Aggregation** — a packet broker / aggregation TAP combines multiple links and feeds one sensor; useful when the "one choke" is really several links.
- **Asymmetric routing** — confirm you see *both* directions of a flow, or stateful tools (Suricata/Zeek) misparse sessions.
- **NAT** — a sensor outside NAT sees the translated (public) address and loses the internal host; inside NAT you keep host attribution. Prefer inside, or collect both and correlate.
- **TLS** — without decryption you still get rich metadata: JA3/JA4 (client/server fingerprints), SNI, certificate subject/issuer/validity, cert chains. Plan to collect these even when you can't see payload.
- **Time** — make sure the sensor's clock is NTP-synced to the same source as the hosts, or correlation across host/network falls apart.

---

6. Host collection — what to request vs deploy

Two categories, and the distinction drives the action:

- **Already generated → request / forward.** The host is already writing these; you just need them shipped (Winlogbeat / Elastic Agent / syslog). Low risk, no new software behavior.
- **Not generated yet → deploy.** You have to add something (Sysmon config, an EDR agent, auditd rules) to produce the telemetry. Higher risk, change control, host impact.

Windows — request first

- **EVTX, the core set** (request + forward to Kibana via Winlogbeat/Elastic Agent):
  - Security (4624/4625 logon, 4648 explicit-cred lateral, 4672 priv, 4688 process w/ cmdline if enabled, 4697 service install, 4698/4702 scheduled task, 4720/4728/4732 account+group changes, 5140/5145 share access, 4768/4769/4771 Kerberos).
  - System (7045 service install, 7035/7036 service control).
  - Application (app crashes, install MSI events).
  - **Sysmon** (if present) — 1 process w/ hashes+cmdline+parent, 3 network, 7 image load, 8 remote thread, 10 process access (LSASS), 11 file create, 13 registry, 22 DNS. Sysmon is the single highest-value Windows host source; if it isn't deployed, that's a **deploy** decision below.
  - **PowerShell** — 4104 script block (deobfuscated script content), 4103 module logging, transcription.
  - Windows Defender / AV operational log, WMI-Activity, TaskScheduler operational, TerminalServices (RDP), Windows Firewall.
- **Dead-box / triage artifacts** (request as a collection or pull with a triage tool — Velociraptor/KAPE if ROE allows): $MFT, USN journal, Amcache/Shimcache, Prefetch, registry hives (SYSTEM/SOFTWARE/SAM/NTUSER), SRUM, Scheduled Tasks XML, Autoruns, browser history, memory image for the live-response cases.
- **Server app logs** — IIS, Apache/Nginx on Windows, DB logs, Exchange, anything web-facing (webshell hunting).

Linux — request first

- **auth** (`/var/log/auth.log` / `secure`), **journald**, and **auditd** (execve, file, network syscalls — auditd is the Linux equivalent of Sysmon in value; if not configured, it's a deploy decision).
- Shell history (`.bash_history` , with timestamps if `HISTTIMEFORMAT` set), **cron**, `/etc/` change evidence, package-manager logs.
- Web/app logs — **Apache/Nginx access+error** (webshell + exploitation), app logs, DB logs.
- Running state for live response — processes, listening sockets, loaded modules, `/proc` , open files.

Ingest note

EVTX and syslog ship cleanly into Kibana via Winlogbeat / Filebeat / Elastic Agent, mapped to ECS. Decide **per source** whether it's worth the ingest cost — ingest budget is a real constraint, and some logs (verbose Application noise) cost more than they return. Priority score (Section 3) applies to log sources

too.

EDR — the deploy decision tree

EDR gives real-time process/network/file/registry telemetry, in-memory behavioral detection, and a response capability (isolate/kill/collect). It's powerful and it's intrusive — a kernel-level agent on someone else's production box. Decide deliberately:

```
For each host (or host group):

  Q1: Is it OT/ICS, embedded, medical, legacy, or an
  |   unsupported/fragile platform?
  |      YES -> DO NOT deploy an agent. Risk to stability/safety
  |             and likely unsupported. Cover it with PASSIVE
  |             NETWORK collection at its segment boundary instead.
  |      NO  -> continue.
  |
  Q2: Does ROE / ownership / change control permit an agent here,
  |   and is the host in scope?
  |      NO  -> request logs only (or passive network). Stop.
  |      YES -> continue.
  |
  Q3: Do you need a RESPONSE capability here (isolate, kill,
  |   remote-collect) or real-time BEHAVIORAL / memory detection
  |   (e.g. cred dumping, injection) that logs won't give you?
  |      YES -> DEPLOY EDR.
  |      NO  -> continue.
  |
  Q4: Is this a high-value / high-likelihood host — DC, jump host,
  |   crown-jewel server, externally exposed, known/suspected
  |   foothold or pivot?
  |      YES -> DEPLOY EDR (start here, expand outward by priority).
  |      NO  -> continue.
  |
  Q5: Does native logging (EVTX+Sysmon / auditd) already cover the
  |   threat adequately for this host's priority?
  |      YES -> request logs; skip the agent (save it for where it
  |             earns its place).
  |      NO  -> DEPLOY EDR.
  |
  Overlay (can override a YES): is a COVERT posture required because
  you're observing a live actor? A new agent install is detectable
  and can tip them off / trigger anti-forensics. If so, weigh
  passive + existing-log collection first and stage EDR for the
  containment phase.
```

When you should NOT deploy EDR — collected in one place:

- OT/ICS/embedded/medical/legacy or any platform where the agent risks stability or isn't supported → passive network only.
- Hosts outside ROE / ownership, or under change control that forbids it.
- When covertness matters more than depth right now (live actor you're watching) — the install is noisy.
- When native logging already covers the threat and the host is low priority — don't spend the agent slot.
- Where it merely duplicates coverage you already have.

Deployment order when you do deploy: crown jewels and likely targets → pivot points (jump hosts, DCs) → externally exposed servers → suspected foothold hosts → expand down the priority list as capacity allows. You rarely blanket-deploy; you deploy by priority, same as everything else in this plan.

---

7. Assumptions and category markers

Every requirement and every sensor/host decision rests on assumptions, and the assumptions differ sharply between a range and a real client. Tag each one so you know what to revalidate when the environment changes:

Marker	Meaning	Why it matters
`[RANGE]`	Holds on the range / exercise network	Synthetic traffic, known build, duplicated images, orchestration and traffic-generator noise, no real production impact, decryption often available. Many "normals" are range artifacts, not real.
`[CLIENT]`	Holds on a real operational network	Unknown build, real users, real production-impact risk, strict ROE, legal boundaries, encryption you can't break, change control. Assumptions here are *unverified until confirmed on site.*
`[BOTH]`	Holds regardless of environment	Platform/protocol truths (AD uses Kerberos, SMB on 445, DNS on 53). Safe to carry across.

Why this is its own section and not a footnote: the single most common way a good plan fails is carrying a range assumption onto a real client (or vice versa). A `[RANGE]` sensor spot that relied on a known flat build may be wrong on a segmented client network; a `[RANGE]` "collect everything, decrypt freely" assumption is often illegal or impossible at a client. The markers are the hand-off contract between environments — when you move, you walk the list and every `[RANGE]` / `[CLIENT]` line gets re-checked; `[BOTH]` lines you keep. (These same markers feed the baseline doc's pre-site model — a `[RANGE]` normal is provisional in a way a `[BOTH]` normal isn't.)

---

8. The collection-requirement record

The atomic unit of the plan. One record per requirement; the fillable worksheet produces these.

Field	Content
ID	CR-001, CR-002 ...
What	The telemetry/data source (e.g. "PowerShell 4104 script block logs").
Where	Terrain location (segment, host group, sensor point).
How	Collection method (SPAN at core, Winlogbeat forward, Sysmon deploy, EDR agent, triage pull).
Why	Technique(s) / engagement it serves — ATT&CK ID(s) or objective. This is the justification you defend to the asset owner.
Priority	likelihood x detection-value x feasibility (Section 3).
Marker	`[RANGE]` / `[CLIENT]` / `[BOTH]` .
Owner	Who executes it (you, host admin, network team).
Status	planned / requested / collecting / failed / gap.
Dependency	What it needs first (decryption, agent install, ROE approval, ingest capacity).

---

9. Fillable worksheet

A reusable worksheet ships alongside this methodology (interactive HTML: add/remove requirement rows, set fields and markers, export to CSV or markdown for the binder). The methodology is the *how-to-think*; the worksheet is the *fill-in-the-blanks* that produces the actual plan for a given network. Use the worksheet on site; keep the filled copy with the engagement binder.

ASCII skeleton (what the worksheet captures):

```
COLLECTION PLAN — <network> — <date> — <analyst>
Engagement: _____________________   Env: [RANGE]/[CLIENT]   Sensors avail: __

TERRAIN SUMMARY
  Segments: ______   Crown jewels: ______   Egress: ______   Core choke: ______

SENSOR PLACEMENT
  #1 location: ______  axis: N-S / E-W  rationale: ______  decrypt: Y/N
  #2 (if any): ______  covers blind spot: ______

REQUIREMENTS
  ID   What                        Where        How          Why(ATT&CK)  Pri  Mark  Owner  Status
  CR1  ____________________________   ___________  ___________  __________   ___  ____  _____  ______
  CR2  ...
```

---

10. Coverage validation and gaps

The plan isn't done when you've listed requirements — it's done when you've proven what you *can't* see. Two validation passes:

- **Against ATT&CK** — lay your planned collection over the technique set from Step 2. Every technique should map to at least one collected data source. Techniques with none are **detection gaps** — log them explicitly ("we are blind to in-memory cred theft on the non-EDR hosts").
- **Against terrain** — walk the diagram. Every segment, crown jewel, and egress should map to a sensor or host source that sees it. Terrain with none is a **visibility gap** ("the OT segment has no host collection and the sensor only sees its boundary").

A named gap is a decision ("accepted — out of ROE") or a to-do ("request second sensor"). An *unnamed* gap is how you get surprised. The gap list is a deliverable in its own right and it feeds straight into the baseline doc: a thing you can't collect is a thing you can't baseline, and both facts belong in the engagement record.

---

11. Quick-start checklist

```
[ ] Pull terrain: diagram, asset inventory, flows, choke points, existing visibility
[ ] Pull/translate threat intel -> ATT&CK techniques  (none? use platform default set)
[ ] Map techniques -> required data sources (host? wire? both?)
[ ] Crosswalk onto terrain -> candidate requirements
[ ] Score each: likelihood x detection-value x feasibility
[ ] Place sensor #1 (egress by default; core if internal engagement); plan #2 to cover its blind spot
[ ] Decide host collection: request existing logs; run the EDR deploy tree per host group
[ ] Write requirement records; tag every one [RANGE]/[CLIENT]/[BOTH]
[ ] Validate coverage vs ATT&CK and vs terrain; log every gap
[ ] Hand the filled worksheet to the binder
```

> **Defensive use.** This methodology is for authorized defensive collection, hunt, and monitoring on networks you are tasked to defend, under ROE. Collection placement, decryption, and agent deployment all have legal and operational boundaries — confirm authority before execution.

Baseline Methodology — "What's Normal"

Purpose. A repeatable way to look at collected data and decide, defensibly, that a given thing is *normal* — so it can be filtered out and your attention goes to what's left. The output is filters; the methodology is how you **earn** each "normal" verdict instead of guessing at it. This is the second stage of the pipeline:

```
[ COLLECT ]  ->  [ BASELINE ]  ->  [ VISUALIZE ]  ->  [ DETECT ]
```

You can only baseline what the collection plan gave you, and everything you confirm as normal becomes a filter the dashboards and detections apply. Get baselining wrong in either direction and you either drown in noise (nothing filtered) or filter away the intrusion (too much filtered). The whole method exists to make that call honest.

---

1. The organizing idea: predict, collect, compare

Most baselining is **reactive** — you stare at what you collected and try to explain it after the fact. That's slow, it's biased toward "looks fine," and it gives you no way to notice the thing that *should* be there and isn't.

Flip it to **predictive**. Before you ever see the data, derive what you *expect* normal to look like from the network build and known environment behavior. Then on site, compare expected against actual. The **difference is the product** — matched traffic is pre-cleared, and your attention lands on the delta.

```
PHASE 0: PREDICT            PHASE 1: COMPARE             PHASE 2: TAG
(pre-site)                  (on-site)                    (ongoing)
build + range + intel        actual collected data        classify each
     |                            |                       observation,
     v                            v                       lock normals,
EXPECTED normal   ---- diff ----> reconcile 3 ways        emit filters
(PROVISIONAL)                  confirm / gap / unknown
```

The discipline that keeps this safe: **a prediction is a hypothesis, not a fact.** Pre-site expectations enter the register as *provisional* and only become *confirmed* after on-site data matches them and they pass one real sanity check. An expectation never silently filters traffic you haven't actually looked at — otherwise the one actor who happens to mimic an expected service gets filtered straight into your blind spot. (This is expectation bias, and Section 8 is about not falling into it.)

---

2. Vocabulary — two kinds of "normal"

Precision here prevents the whole method from collapsing into "stuff I've seen before":

- **Known-good** — normal you can *affirmatively justify*: it maps to a documented baseline item, a range/range artifact, an expected protocol for that terrain, or sanctioned automation. You can say *why* it's normal. Only known-good gets filtered.
- **Unknown-benign** — traffic that *looks* harmless but you can't yet justify. It has not earned "normal." It stays visible and goes to triage/hunt until it is either justified (→ known-good) or found hostile.

The failure mode is treating unknown-benign as known-good because it's quiet and you're busy. The gates in Section 5 exist precisely to stop that.

---

3. Phase 0 — Pre-site: build the expected baseline (provisional)

Before arrival, derive what normal *should* be from everything you can know without touching the live network. Output: an **expected-normal register** (every item marked `provisional`) plus an **expected sensor-coverage map** (what each planned sensor/host source should be seeing if the collection plan is right).

Sources for the prediction (most also fed the collection plan — shared inputs):

- Network diagram + asset inventory → expected hosts, roles, segments, and the flows that *must* exist (DC replication, DNS to the resolver, workstations to file servers, egress via the proxy).
- Service/port matrix + authorized-software list → expected protocols and ports per segment.
- Known management/monitoring tooling → expected admin traffic (WSUS/SCCM, patch, backup, vuln scanner sweeps, monitoring beacons).
- **range build / range docs + traffic-gen (e.g. a synthetic-user generator) configs** → expected synthetic users, orchestration traffic, scoring server/grading traffic, injected "internet" (see Section 6).

- Prior-engagement baselines for the same environment → start from last time's confirmed normals (re-marked provisional until re-confirmed).
- Vendor / protocol defaults → expected chatter (NTP, ARP, NetBIOS, mDNS, DHCP, CRL/OCSP, telemetry).

For each expected item, record: the observable (host/flow/protocol/process), *why* you expect it (which source predicted it), the expected sensor that should see it, a confidence, and the environment marker (`[RANGE]` / `[CLIENT]` / `[BOTH]` — a `[BOTH]` prediction is higher-confidence than a `[RANGE]` one). State = `provisional` .

The act of doing this before arrival is half the value even before you compare: it forces you to actually read the build, and it surfaces *expected* telemetry you'll then check is arriving (a coverage check you otherwise skip).

---

4. Phase 1 — On-site: ground truth and 3-state reconciliation

Capture the actual collected data, then diff it against the expected baseline. Every observable lands in exactly one of three states:

```
                      OBSERVED on site?
                  yes                no
          +--------------------+--------------------+
EXPECTED  | CONFIRM            | GAP / ANOMALY      |
  yes     | matches prediction | expected but ABSENT:|
          | -> sanity-check    | coverage gap, down  |
          | once, then promote | service, blocked    |
          | provisional->      | collection, or a    |
          | confirmed          | real change. INVEST.|
          +--------------------+--------------------+
  no      | UNKNOWN            |   (n/a)            |
          | not predicted:     |                    |
          | ism? new normal?   |                    |
          | threat? -> TRIAGE  |                    |
          +--------------------+--------------------+
```

- **CONFIRM (expected + observed).** Run the prediction through one real check the first time (is it actually the WSUS server, actually the traffic-generator user pattern?), then promote `provisional -> confirmed` . Now it can emit a filter.
- **GAP / ANOMALY (expected + absent).** The sleeper value of the whole method. Something you were sure you'd see isn't there. Three causes, all worth knowing: (a) a **collection/coverage gap** — the sensor or agent isn't seeing what it should, your collection plan has a hole; (b) a **down/blocked service** — operationally relevant; (c) a **real change or incident** — something altered the environment. Pure reactive baselining never surfaces this cell, because you can't notice the absence of a thing you weren't expecting.
- **UNKNOWN (observed + unexpected).** Not predicted. Route through the tagging gates (Section 5): is it a range-ism? a legitimately new normal you add to the baseline? or a hunt lead? This is where real findings hide.

The reconciliation is itself an analytic deliverable — the confirm/gap/unknown breakdown tells the engagement lead the state of visibility and the size of the hunt surface on day one.

---

5. Phase 2 — The tagging workflow (earning "normal")

For any observation that needs a verdict (every UNKNOWN, and anything new that appears later), run it through escalating gates. The first gate that justifies it wins; if none do, it stays visible and goes to hunt. Order matters — cheapest/most-certain justification first.

```
Observation needing a verdict
      |
  GATE 1: Is it a known range ism? (Section 6 catalog)
      |      YES -> known-good [RANGE]. Tag, note the ism, emit filter (scoped to range).
      |      NO  -> down.
      |
  GATE 2: Does it match a DOCUMENTED baseline item (confirmed register,
      |   prior engagement, asset/service matrix)?
      |      YES -> known-good. Tag, link the register ID, emit filter.
      |      NO  -> down.
      |
  GATE 3: Is it an EXPECTED protocol/flow for THIS terrain?
      |   (DNS from host to resolver, SMB host->file server, Kerberos to DC,
      |   NTP to time source, the documented egress path)
      |      YES -> known-good, but VERIFY endpoints/direction match terrain
      |             (right source, right destination, right port, right volume).
      |             Confirmed -> emit filter. Mismatch -> treat as UNKNOWN, down.
      |      NO  -> down.
      |
  GATE 4: Is it sanctioned/benign AUTOMATION?
      |   (patch, backup, vuln scan, SCCM/WSUS, monitoring, scoring server)
      |      YES -> VERIFY it's the real source/schedule/volume, then known-good,
      |             emit filter.
      |      NO  -> down.
      |
  NO GATE PASSED -> UNKNOWN-BENIGN at best. Do NOT filter.
      -> keep visible, send to hunt / triage as a lead.
```

Each gate has a *verify* step for a reason: the gate tells you a benign explanation is *plausible*; the verify step confirms the traffic actually *is* that thing and not an actor wearing its clothes (a "WSUS" flow to the wrong host, SMB in the wrong direction, a "backup" at the wrong hour). Plausible is not confirmed.

---

6. Range-ism catalog (Gate 1 content)

Range networks generate traffic that looks strange or alarming but is an artifact of how the range is built. Know these cold so you filter them fast — and so you *don't* mistake a real technique for one. (Draft list; prune/extend per the specific range build.)

Ism	What you see	How to confirm it's the range, not a threat
Range NAT / jump hosts	Many internal hosts sourcing from one address; odd gateway hops	Matches the documented range NAT/jump layout; address is the known range infra
Orchestration traffic	Control-plane chatter to a management host (provisioning, config push)	Destination is the known orchestration host; regular, structured, from the build docs
traffic-generator / synthetic users	"User" activity that's too regular — same sites, same intervals, 24/7, no human rhythm	Matches the traffic-generator config; machine-perfect periodicity; originates from the seeded user hosts
Cloned-image duplicates	Duplicate hostnames, duplicate MACs, same machine SIDs/GUIDs across hosts	Images were cloned from one golden build; expected on a range, alarming on a client
INetSim / simulated internet	"Internet" services all resolving to one internal responder; every domain answers, often same IP	DNS/HTTP all funnel to the known simulation host; real internet doesn't behave this uniformly
PXE / imaging traffic	TFTP/DHCP/PXE bursts, large image pulls at range reset	Coincides with range build/reset windows; to/from the imaging server
Monitoring beacons	Periodic check-ins to a monitoring/collection host	Destination is the known monitoring stack; your own tooling
Scoring server / grading traffic	Regular polling to a scoring host, health checks	Destination is the exercise scoring server; part of the exercise, not the scenario

Why this is a gate and not just a note: these are the highest-volume false alarms on a range, and every one of them has a real-world lookalike (NAT vs C2 aggregation, traffic-generator vs beaconing, INetSim vs sinkhole, cloned MACs vs spoofing). Clearing them *first*, by matching the documented build, both drops the noise floor and trains you to verify rather than assume. Every `[RANGE]` filter is scoped to the range and dropped when you move to a client — this ties directly to the `[RANGE]` / `[CLIENT]` / `[BOTH]` markers from the collection plan.

---

7. The Baseline Register

The durable record of every "normal" verdict. One row per observable; the `state` column drives the predict→confirm lifecycle.

Column	Content
ID	BL-001 …

Column	Content
Date	When recorded / last reviewed
Scope	Segment / host group / sensor it applies to (filters are scoped, never global-by-accident)
Observable	The thing: host, flow (src/dst/port/proto), process, domain, JA3, user pattern
Class	network / host-process / host-auth / dns / tls / automation / user
State	`provisional` (predicted, pre-site) → `confirmed` (observed + verified) → `retired` (no longer valid)
Justification	*Why* it's normal: ism name, register link, terrain expectation, automation source. No justification → it doesn't belong here.
Marker	`[RANGE]` / `[CLIENT]` / `[BOTH]`
Filter	The emitted filter (KQL / Suricata suppress / Zeek intel exclusion / Defend event filter) — see Section 9
Analyst	Who made the call
Review-by	When to re-verify (nothing is normal forever)

Rules: only `confirmed` items emit active filters. `provisional` items are visible-but-flagged, never filtered. A filter always carries its register ID so anyone can trace *why* traffic is being suppressed back to a justification — a filter with no traceable reason is how intrusions get hidden.

---

8. Guardrails — not fooling yourself

The method's power (pre-cleared expectations) is also its danger (you see what you expect). Hard rules:

- **Provisional never filters.** A pre-site prediction is a lead to confirm, not a license to suppress. Nothing gets filtered until it's `confirmed` with a verified justification.
- **Confirm = observed + one real check.** Promotion isn't automatic on a match; the first match gets a sanity check against the actual endpoints/volume/timing.
- **Plausible is not confirmed.** Every gate's verify step exists because a benign *category* (WSUS, backup, traffic-generator) is exactly what a smart actor hides inside.
- **Filters are scoped and sourced.** Every filter has a scope (segment/host) and a register ID. No global suppressions, no orphan filters.
- **Absence is a finding.** The GAP cell is not "nothing to do" — expected-but-absent gets investigated like any anomaly.
- **Re-verify on a clock.** `review-by` on every row. A normal from three weeks and one patch cycle ago may be stale.

---

9. Filter emission (hand-off to the Detection pack)

A confirmed register item becomes a concrete filter in whatever stack carries that data. The exact syntax lives in the **Detection & Filter Pack**; the mapping is:

Register class	Emits to	Form
network flow / host / port	Kibana saved filter; Zeek intel exclusion	KQL `NOT (...)` ; Zeek script/notice suppression
Suricata-visible pattern	Suricata	`suppress` / `threshold` by sig+host, or scoped rule
host process / parent-child / user pattern	Elastic Defend	**event filter** (drop at agent) or **exception** (on a rule)
DNS / TLS (JA3, SNI)	Kibana / Zeek	KQL filter; Zeek intel-framework exclusion

Baseline filtering is applied **first** in the dashboards and detections — confirmed-normal drops out before analysis — so the noise floor set here directly determines the signal downstream. This is why the register's discipline matters: a sloppy "normal" here silently blinds every dashboard and rule that inherits the filter.

---

10. Quick-start checklist

```
PRE-SITE (Phase 0)
[ ] Read the build: diagram, assets, service matrix, range/traffic-generator docs, prior baselines
[ ] Derive expected normals -> register, every row state=provisional, marked [RANGE]/[CLIENT]/[BOTH]
[ ] Build expected sensor-coverage map (what each source SHOULD see)

ON-SITE (Phase 1)
[ ] Capture actual collected data
[ ] 3-state diff vs expected: CONFIRM / GAP-ANOMALY / UNKNOWN
[ ] Work the GAP cell: coverage hole? down service? real change?
[ ] Promote confirmed (observed + sanity check): provisional -> confirmed

ONGOING (Phase 2)
[ ] Run every UNKNOWN through Gates 1-4; verify, don't assume
[ ] Known-good -> register + scoped, sourced filter; unknown-benign -> stays visible, to hunt
[ ] Set review-by on every row; re-verify on schedule
[ ] Drop all [RANGE] filters when moving to a client network
```

> **Defensive use.** Baselining determines what gets filtered out of defensive analysis. The guardrails exist so that filtering never hides hostile activity. Applied as written — justify every normal, scope every filter, treat absence as a finding — the method reduces noise without reducing coverage.

Kibana Dashboards — Host + Network

Purpose. Standard dashboards that let you triage a host and a network view fast, from a known-good / known-bad posture. This is the third pipeline stage — the visualize layer that sits on top of what you collected and after baseline filtering removes the confirmed-normal:

```
[ COLLECT ] -> [ BASELINE ] -> [ VISUALIZE ] -> [ DETECT ]
```

A dashboard shows only what collection delivered and baseline left behind. Build these to mirror that pipeline, not as a wall of pretty charts.

---

1. Organizing idea: funnel, and one question per panel

Two rules shape every dashboard here:

1. **Each panel answers one hunting question.** Not "show me network traffic" — "which internal hosts are beaconing to an external destination on a fixed interval?" If you can't state the question a panel answers, cut it. A panel with no question is decoration, and decoration is where findings hide.
2. **Layout is a funnel: overview → drilldown.** Top of the dashboard is the widest, cheapest triage (counts, top-N, timelines) so you can see the shape of the data in one screen. Lower panels get narrower and more specific. You scan the top, something looks off, you click down. This matches how you actually hunt and it keeps the dashboard usable on a bad-bandwidth day on site.

Three dashboards, in funnel order: **Overview** (landing, both axes) → **Host** → **Network**. Baseline filters (from the baseline doc's register) are applied as pinned filters so confirmed-normal is already gone before you look.

---

2. Data model assumptions (ECS + indices)

Built on Elastic Common Schema (ECS) so host and network fields line up for correlation. Assumed sources/indices (adjust to the stack on site):

Source	Index pattern / data view	Key ECS fields
Windows events / Sysmon (Elastic Agent or Winlogbeat)	`logs-windows.*` , `winlogbeat-*`	`event.code` , `process.name` , `process.command_line` , `process.parent.name` , `user.name` , `winlog.event_data.*` , `host.name`
Linux auditd / journald (Elastic Agent)	`logs-system.*` , `logs-auditd.*`	`process.*` , `user.name` , `event.action` , `host.name`
Elastic Defend (endpoint)	`logs-endpoint.events.*` , `.alerts-security.*`	`event.category` , `process.*` , `destination.ip` , `dll.*` , `event.type`
Zeek (via Filebeat zeek module)	`logs-zeek.*` , `filebeat-*`	`zeek.conn.*` , `zeek.dns.*` , `zeek.ssl.*` , `zeek.http.*` , `source.ip` , `destination.ip` , `network.bytes`
Suricata (eve.json via Filebeat)	`logs-suricata.*`	`suricata.eve.alert.*` , `event.severity` , `source.ip` , `destination.ip` , `network.protocol`
Packetbeat (if used)	`packetbeat-*`	`network.*` , `dns.*` , `tls.*` , `source/destination.*`

Mechanics: build **data views** per source (or one `logs-*` + the `.alerts-*` view). Where a field you want isn't mapped, add a **runtime field** (e.g. parse a domain's length, extract a registry path). Queries below are **KQL** (Kibana Query Language) unless noted; the Detection pack has the ES|QL / Lucene variants and the Zeek/Suricata rule text.

---

3. Overview dashboard (landing)

Widest triage, both axes, built to be read in one screen before coffee:

Panel	Question	Viz	Source/fields	Query / notes	Normal looks like	Pivot on
Ingest health	Is every expected source still arriving?	Area, split by `event.module` / `data_stream.dataset` over time	all	count over time per dataset	steady lines per source	a line that drops = collection GAP (ties to baseline Phase 1 GAP cell)
Alert volume	Where are detections firing?	Bar, split by `signal.rule.name` / `suricata.eve.alert.signature`	`.alerts-*` , suricata	last 24h	low, known-noise sigs	a spike or a new sig
Top talkers	Who moves the most?	Bar, `source.ip` by `sum(network.bytes)`	zeek.conn	exclude baseline	internal servers you expect	an unexpected host high on the list
Egress bytes-out	What's leaving?	Bar, `destination.ip` by bytes where `destination.ip` is external	zeek.conn	direction = outbound	known SaaS/update dests	a high-bytes unknown external dest

Panel	Question	Viz	Source/fields	Query / notes	Normal looks like	Pivot on
New hosts	Anything appear that wasn't predicted?	Data table, unique `host.name` / `source.ip` not in baseline	all	compare to Baseline Register	nothing new	a host = UNKNOWN (baseline Gate it)

4. Host dashboard

Depth on endpoints. Funnel: process activity → persistence → identity → host-network. Each panel is a known hunting technique.

Panel	Question	Viz	Source / fields	Query (KQL)	Normal	Pivot
Process timeline	What's executing, when?	Histogram split by `process.name`	Sysmon 1 / 4688	`event.code:("1" or "4688")`	steady business apps	burst of rare/new names
Rare processes	What runs almost nowhere?	Data table, `process.name` by count asc, split by `host.name`	Sysmon 1	`event.code:"1"` + sort rare	common binaries only	a binary on 1-2 hosts
Parent-child anomalies	Spawns that shouldn't happen	Table, `process.parent.name` → `process.name`	Sysmon 1	`process.parent.name:("winword.exe" or "excel.exe" or "outlook.exe") and process.name:("cmd.exe" or "powershell.exe" or "wscript.exe" or "mshta.exe")`	empty	any hit (Office spawning a shell)
LOLBins	Living-off-the-land binaries	Table, `process.name` + `process.command_line`	Sysmon 1	`process.name:("rundll32.exe" or "regsvr32.exe" or "mshta.exe" or "certutil.exe" or "bitsadmin.exe" or "wmic.exe")`	known admin use	odd args (URLs, encoded, temp paths)
Encoded PowerShell	Obfuscated PS	Table, `process.command_line`	Sysmon 1 / 4104	`process.name:"powershell.exe" and process.command_line:(*enc* or *-e* or *FromBase64* or *IEX* or *DownloadString*)`	none/known jobs	any encoded one-liner
Script block content	What did PS actually run?	Table, `powershell.file.script_block_text`	4104	`event.code:"4104"`	admin scripts	obfuscation, download cradles
Persistence	New autostart	Table, event by `host.name`	7045/4697/4698/4702, Sysmon 13	`event.code:("7045" or "4697" or "4698" or "4702")`	planned installs	unscheduled new service/task
Logon activity	Who authenticated, how?	Table, `winlog.event_data.LogonType`, `user.name`, `source.ip`	4624/4625	`event.code:("4624" or "4625")`	type 2/3/5 from known hosts	type 3/10 spraying, off-hours admin
Lateral (explicit creds)	Admin reuse across hosts	Table, `user.name` x `host.name`	4648	`event.code:"4648"`	few admin paths	one account hitting many hosts
Priv use	Privilege escalation markers	Table	4672	`event.code:"4672"`	known admins	a workstation user getting priv
Host-network by process	What is a process talking to?	Table, `process.name` → `destination.ip`	Sysmon 3 / Defend	`event.category:"network"`	browser/updater to known	lsass/odd proc making external conns
EDR alerts	Behavioral detections	Table, `signal.rule.name`, `host.name`	`.alerts-*`	all	low known-noise	malware/injection/ransomware class

5. Network dashboard

Breadth on the wire. Funnel: connection shape → DNS → TLS → HTTP → alerts. Encryption-aware (metadata when you can't see payload).

Panel	Question	Viz	Source / fields	Query	Normal	Pivot
Connection overview	Shape of all traffic	Histogram by `network.protocol`	zeek.conn	all	expected protocol mix	a protocol that shouldn't be there

Panel	Question	Viz	Source / fields	Query	Normal	Pivot
Top talkers / listeners	Who dominates?	Bars, `source.ip` / `destination.port` by bytes+count	zeek.conn	exclude baseline	servers + proxy	an unexpected heavy host
Egress destinations	Where does outbound go?	Table, external `destination.ip` / `.as.organization` by bytes	zeek.conn	outbound only	CDNs, update, SaaS	unknown ASN / raw IP / high bytes
Long connections	Sessions that never end	Table, `zeek.conn.duration` desc	zeek.conn	`zeek.conn.duration > 3600`	backups, DB, RDP sessions	long conn to an external unknown
Beaconing	Fixed-interval check-ins	Table/Lens, group `source.ip` + `destination.ip`, look at interval regularity of `@timestamp`	zeek.conn	see Detection pack (interval analysis)	traffic-generator (ism), NTP, update polls	regular small conns to one external dst
Rare destinations	First/seldom-seen externals	Table, `destination.ip` by count asc	zeek.conn	rare sort	none new	a brand-new external endpoint
DNS volume + NXDOMAIN	Resolution anomalies	Histogram `dns.question.name`; table `dns.response_code:"NXDOMAIN"`	zeek.dns	all	known domains	NXDOMAIN spikes (DGA), TXT floods
Long / rare domains	Tunneling + DGA markers	Table, runtime field `domain length`; `dns.question.type:"TXT"` count	zeek.dns	`length > 50` or high-entropy	normal FQDNs	long/high-entropy names, TXT tunneling
TLS fingerprints	Client/cert anomalies	Table, `tls.client.ja3` / `ja4`, `tls.server.x509.issuer`, `tls.server.ja3s`	zeek.ssl/x509	all	known JA3 set, real CAs	rare JA3, self-signed, odd SNI vs cert
SNI vs destination	Domain fronting / mismatch	Table, `tls.client.server_name` x `destination.ip`	zeek.ssl	all	SNI matches expected IP/ASN	SNI not matching the endpoint
HTTP UA + URI	Web badness	Table, `http.user_agent.original`, `url.path`	zeek.http	all	browser/known UAs	rare UA, suspicious URIs, webshell paths
Suricata alerts	Signature hits	Table, `suricata.eve.alert.signature` by `event.severity`	suricata	all	tuned known-noise	high-sev or new category

6. Build mechanics

- **Data views / index patterns** — create per source (`logs-windows.*`, `logs-zeek.*`, `logs-suricata.*`, `.alerts-*`) or a combined `logs-*`. Set the time field (`@timestamp`).
- **Runtime fields** — add for anything not mapped: domain length/entropy, extracted registry path, parsed command-line token, bytes ratio (out/in). Define on the data view so every panel can use them.
- **Saved searches** — build the heavy KQL once as a saved search, then base panels on it; keeps queries consistent and editable in one place.
- **Panels** — Lens for most (fast), TSVB for time-series/interval work (beaconing), Data Table for triage lists. Give each panel a title that *states its question*.
- **Pinned / dashboard-level filters** — apply the **baseline filters** here (Section 7) so confirmed-normal is removed dashboard-wide. Pin the environment filter (`[RANGE]` range infra out) at the dashboard level.
- **Drilldowns** — wire panel drilldowns: click a `host.name` in Network → open the Host dashboard filtered to that host; click a `destination.ip` → open a destination-detail dashboard. This is the funnel made clickable and it's where triage speed comes from.
- **Time + refresh** — default a short relative window (last 24h) with a quick-select; don't auto-refresh aggressively on a constrained sensor.
- **Saved objects / portability** — export the dashboards + data views + saved searches as an **NDJSON saved-objects bundle** so the whole pack imports onto the next stack in one step. Keep that bundle in the engagement binder alongside these docs. Watch index-pattern name drift between environments — re-point data views on import.

7. Baseline integration (filters applied first)

The dashboards inherit the Baseline Register. A `confirmed` register item with a KQL filter becomes a **dashboard-level negative filter**, so the confirmed-normal is gone before any panel aggregates. Pattern:

```
Baseline Register (confirmed, [RANGE]/[CLIENT]/[BOTH])
        |
        v   emit KQL
Dashboard-level filters (pinned):
   NOT (source.ip: <range NAT/jump>)              // BL-003 [RANGE] ism
   NOT (destination.ip: <WSUS> and destination.port: 8530)  // BL-011 automation
   NOT (tls.client.ja3: <known-good JA3 set>)     // BL-020 confirmed
        |
        v
Panels aggregate on what's LEFT  =  the hunt surface
```

Rules carried from the baseline doc: only `confirmed` items filter; every filter names its `BL-ID`; drop all `[RANGE]` filters when the dashboards move to a client. A dashboard quietly filtering on an unsourced rule is the same failure as an unsourced register row — the filter list is auditable or it's dangerous.

> **Defensive use.** These dashboards visualize defensive telemetry for hunt and triage on authorized networks. The baseline filters reduce noise; keep every filter traceable to a Baseline Register justification so visualization never hides activity.

Intelligence Analysis Methodology — Intel to Action

Purpose. Convert a piece of threat intelligence into a defined analytic action: the data to examine, the query to run, and the evidence that confirms or clears it. Intel names a behavior; this methodology resolves that behavior to a record in collected data. It sits between collection and detection and is applied during hunt and triage:

```
[ COLLECT ]  ->  [ BASELINE ]  ->  [ VISUALIZE ]  ->  [ DETECT ]
                        \                              /
                         \                            /
                          ----->  [ INTEL -> ACTION ]  <-----
                            technique / indicator -> data source -> query -> verdict
```

An intel input is a tasking, not a conclusion. The deliverable of this process is a verdict — present, absent, or not collectable — backed by a query result and a second corroborating source.

---

1. The organizing idea: every technique writes to a known place

A technique is an action taken on a host or a network. Every action that matters leaves a record in a specific data source. ATT&CK already names that data source for each technique as a *data component* — process creation, command execution, network traffic flow, logon session, file modification, Windows registry key modification, and so on. The collection plan already mapped those data components to concrete telemetry: Sysmon, forwarded EVTX, Zeek, Suricata, EDR, memory, and disk artifacts.

The intel-to-action path is the composition of those two mappings:

```
TECHNIQUE (T-code)
     |  ATT&CK: technique -> data component(s)
     v
DATA COMPONENT  (e.g. "Command Execution")
     |  collection plan: data component -> collected telemetry
     v
COLLECTED TELEMETRY  (e.g. Sysmon 1 + PS 4104 in Kibana)
     |  this doc: telemetry -> query
     v
QUERY  ->  RESULT  ->  VERDICT (+ corroboration)
```

The analyst does not improvise where to look. The technique dictates the data component, the data component dictates the telemetry, and the telemetry dictates the query. When any link is missing, the gap is explicit and named, not discovered by failing to find evidence.

---

2. Classify the intel input

Intel arrives in five forms. The form sets the entry point into the pipeline.

Input	Example	Entry point
Technique ID	`T1059.001`	Section 4 crosswalk — the primary path
Tactic only	"credential access reported"	Expand to the technique set for that tactic (Section 7), run each
Procedure / tool	"actor uses PsExec", "Cobalt Strike"	Resolve the tool to its technique(s), then the crosswalk
Atomic indicator	hash, IP, domain, filename	Section 6 indicator crosswalk
Behavioral report	"beaconing to a rare host every 60s"	Describe-to-technique, then the crosswalk

Record the input verbatim and its source and date before acting. Stale intel and precise intel are both actionable, but they are weighted differently at the reporting stage.

Normalize every input to one or more technique IDs where possible. A tool name is not a tasking until it is resolved to the techniques it performs. PsExec is `T1569.002` and `T1021.002`. Mimikatz is `T1003.001`. The resolution is the first analytic step, and it is recorded.

---

3. The reference process

Run the same loop for every technique under examination.

```
1 SCOPE       Which hosts/segments could exhibit this technique? Restrict the query to them.
2 MAP         Technique -> data component -> collected telemetry (Section 4 table).
3 QUERY       Run the mapped query over the incident window.
4 EVALUATE    Hit / miss / not-collectable. A miss is only valid if the source was collected.
5 CORROBORATE A hit requires a second independent source before it is a finding.
6 PIVOT       A confirmed technique implies adjacent techniques (Section 7). Queue them.
7 RECORD      Technique, query, result, corroboration, verdict, confidence (Section 8).
```

Two rules govern the loop:

- **A miss is not a clear unless the source was collected.** "No 4104 events" means the technique is absent only if Script Block logging was on and forwarded. If it was not, the verdict is *not collectable*, which is a collection gap, not a negative finding.
- **A hit is a lead until corroborated.** One source places a technique; a second independent source confirms it. The corroboration column in Section 4 names the second source for each technique.

---

4. Technique-to-data crosswalk

Each entry gives: what the technique writes, where it was collected, the query to run, and the independent source that corroborates a hit. Queries are Kibana KQL against the collected index unless marked. Replace the window and host placeholders. Host-live and memory equivalents are given where the collected telemetry may be absent.

Execution

T1059.001 — PowerShell

- Writes: command execution, script block text, process creation.
- Collected: PowerShell/Operational 4104 and 4103; Sysmon 1; Prefetch `POWERSHELL.EXE-*.pf`.
- Query:

```
event.code:4104 and powershell.file.script_block_text:(*FromBase64String* or *IEX* or *Invoke-Expression* or *DownloadString* or *Net.WebClient* or *-enc* or *-ec* or *EncodedCommand*)
```

```
event.code:1 and process.name:powershell.exe and process.command_line:(*-enc* or *-w hidden* or *-nop* or *bypass*)
```

- Corroborate: 4104 script text against the Sysmon 1 command line for the same PID and time; a network child from the same PID.
- Common benign: SCCM, Intune, admin automation from management hosts — confirm source host against the baseline before clearing.
- Pivot: parent of the PowerShell process (`T1566` phishing document, `T1059` chain), network destination (`T1071`), files written (`T1105`).

T1059.003 — Windows Command Shell

- Collected: Sysmon 1; 4688 if process-creation auditing with command line is on.
- Query:

```
event.code:1 and process.parent.name:(winword.exe or excel.exe or outlook.exe or w3wp.exe or httpd.exe) and process.name:(cmd.exe or powershell.exe)
```

- Corroborate: parent process document or service, and any file the child wrote.

Persistence / Privilege Escalation

T1053.005 — Scheduled Task

- Writes: scheduled job creation, file and registry modification.
- Collected: Security 4698 create, 4702 update, 4699 delete; Sysmon 1 (`svchost.exe -k netsvcs` or `taskeng.exe` spawning the action); disk `C:\Windows\System32\Tasks\` ; registry `...\TaskCache\Tree` .
- Query:

```
event.code:(4698 or 4702) and not winlog.event_data.TaskName:("\\Microsoft\\*")
```

- Corroborate: the task XML on disk against the 4698 record; the action binary against a process-creation event at the task's trigger time.
- Pivot: the action binary (`T1059` , `T1105`), the account that created the task (`T1078`).

T1547.001 — Registry Run Keys / Startup Folder

- Collected: Sysmon 13 registry value set; Security 4657 if registry auditing is on; disk Run keys and Startup folders; Autoruns output.

- Query:

```
event.code:13 and registry.path:(*\\CurrentVersion\\Run* or *\\CurrentVersion\\RunOnce* or *\\CurrentVersion\\Explorer\\Shell Folders*)
```

- Corroborate: the referenced binary against a process-creation event at boot or logon.
- Pivot: the referenced binary and its signer.

T1543.003 — Windows Service

- Collected: System 7045 service installed; Security 4697; Sysmon 13 under `...\Services\`.
- Query:

```
event.code:7045 and not winlog.event_data.ImagePath:("C:\\Windows\\*" or "C:\\Program Files*")
```

- Corroborate: the service image path against a process-creation and the on-disk file hash.
- Pivot: `T1569.002` service execution, lateral origin if the service was created remotely.

Defense Evasion

T1055 — Process Injection

- Writes: process access, remote thread creation; the injected region is only in memory.
- Collected: Sysmon 8 CreateRemoteThread, Sysmon 10 ProcessAccess; memory image.
- Query:

```
event.code:8 or (event.code:10 and winlog.event_data.TargetImage:(*lsass.exe or *explorer.exe or *svchost.exe))
```

- Memory: `windows.malfind` for private committed RWX regions; `windows.ldrmodules` for unlinked modules.
- Corroborate: Sysmon 8/10 source process against a `malfind` hit in the same process in the memory image.
- Pivot: the source process (its origin technique), the target process network and handles.

T1070.001 — Clear Windows Event Logs

- Collected: Security 1102; System 104.
- Query:

```
event.code:(1102 or 104)
```

- Corroborate: a gap in the cleared log's continuity against the 1102/104 timestamp; the account that performed the clear.
- Note: a log clear is itself the indicator. Treat the window after it as reduced-visibility and lean on memory and disk.

T1070.006 — Timestomp

- Collected: `$MFT`, USN journal.
- Query (disk, not Kibana):

```
# MFT: compare $STANDARD_INFO vs $FILE_NAME timestamps per record
MFTECmd.exe -f $MFT --csv out ; # then filter records where SI < FN or SI sub-second = .0000000
```

- Corroborate: an execution artifact (Prefetch, Amcache first-seen) that contradicts the file's standard-information time.

Credential Access

T1003.001 — LSASS Memory

- Collected: Sysmon 10 ProcessAccess to lsass; Security 4656/4663 on lsass if object auditing is on; Sysmon 1 for dump tooling.
- Query:

```
event.code:10 and winlog.event_data.TargetImage:*lsass.exe and winlog.event_data.GrantedAccess:("0x1010" or "0x1410" or "0x143a" or "0x1438")
```

```
event.code:1 and process.command_line:(*comsvcs.dll*MiniDump* or *procdump*lsass* or *-ma*lsass*)
```

- Corroborate: the accessing process against its origin, and any `.dmp` file written (Sysmon 11).
- Pivot: the account whose credentials were exposed, then lateral movement under that account (`T1021`).

Lateral Movement

T1021.001 — Remote Desktop

- Collected: Security 4624 type 10, 4778/4779; Sysmon 3; TerminalServices-RemoteConnectionManager.
- Query:

```
event.code:4624 and winlog.event_data.LogonType:10
```

- Corroborate: the source IP against a corresponding outbound RDP connection on the source host; the account against its normal logon hours.

T1021.002 — SMB / Windows Admin Shares

- Collected: Security 4624 type 3, 4648 explicit credentials, 5140/5145 file share access.
- Query:

```
event.code:4624 and winlog.event_data.LogonType:3 and winlog.event_data.AuthenticationPackageName:NTLM and not user.name:(*$)
```

```
event.code:5145 and winlog.event_data.ShareName:(*\\ADMIN$ or *\\C$)
```

- Corroborate: one source account reaching many destinations in a short window; a service or scheduled task created at the destination immediately after (`T1569.002` , `T1053.005`).
- Pivot: fan-out from the source account defines the spread; scope every destination.

Command and Control

T1071.001 — Web Protocols

- Collected: Zeek `conn` , `http` , `ssl` ; Suricata; proxy logs.
- Query:

```
event.dataset:zeek.conn and destination.port:(80 or 443) and not destination.ip:<known_good_cidr>
```

- Analysis: group by `destination.ip` and derive connection interval and byte symmetry. A fixed interval with small, regular request sizes is a beacon. Irregular timing and asymmetric volume is normal browsing.
- Corroborate: the beaconing host's Sysmon 3 for the owning process; the process against Q1 of the host triage.
- Pivot: the owning process (its persistence and origin), the destination across all hosts.

T1071.004 — DNS

- Collected: Zeek `dns` ; EVTX DNS client; firewall DNS.
- Query:

```
event.dataset:zeek.dns and (dns.question.name:/.{50,}/ or dns.response_code:NXDOMAIN)
```

- Analysis: high query volume to one parent domain, long or high-entropy subdomain labels, sustained NXDOMAIN — DNS tunneling or domain generation.
- Corroborate: the querying host's process holding the resolver socket.

T1105 — Ingress Tool Transfer

- Collected: Zeek `files` / `http` ; Sysmon 11 file create plus Sysmon 3 network from the same process.
- Query:

```
event.code:11 and file.path:(*\\Temp\\* or *\\AppData\\* or *\\ProgramData\\* or *\\Public\\*) and file.extension:(exe or dll or ps1 or bat or scr)
```

- Corroborate: the Sysmon 3 network event that preceded the file write from the same PID; the file hash against intel.

Impact

T1486 — Data Encrypted for Impact

- Collected: Sysmon 11 high-rate file creation; Security 4663; Sysmon 1 for shadow-copy deletion.
- Query:

```
event.code:1 and process.command_line:(*vssadmin*delete*shadows* or *wmic*shadowcopy*delete* or *bcdedit*recoveryenabled*no*)
```

- Corroborate: a burst of file writes with a common new extension from one process against the shadow-deletion command from the same host.

Linux

T1053.003 — Cron

- Collected: auditd execve; `/etc/cron*`, `/var/spool/cron/`; syslog cron facility.
- Query (host):

```
for f in /etc/cron* /var/spool/cron/crontabs/*; do echo "== $f"; cat "$f" 2>/dev/null; done
```

- Corroborate: the cron command against an auditd execve at the scheduled time.

T1543.002 — Systemd Service

- Collected: auditd; unit files under `/etc/systemd`, `/lib/systemd`, `~/.config/systemd`; journald.
- Query (host):

```
find /etc/systemd /lib/systemd /usr/lib/systemd /home/*/.config/systemd -name '*.service' -mtime -14 -ls 2>/dev/null
```

- Corroborate: the unit `ExecStart` binary against a running process and its on-disk hash.

T1505.003 — Web Shell

- Collected: web-root file creation; web access log; auditd; Zeek `http`.
- Query (host):

```
find /var/www -type f -name '*.php*' -mtime -30 -printf '%TY-%Tm-%Td %p\n' | sort
```

```
grep -RliE 'eval\(|base64_decode|system\(|assert\(|passthru\(|\$_(GET|POST|REQUEST)\[' /var/www 2>/dev/null
```

- Corroborate: a POST in the access log to the suspect path at or after its file-creation time.
- Pivot: the web process's child shells (`T1059`) and outbound sockets (`T1071`).

T1098.004 — SSH authorized_keys

- Collected: file modification; auditd watch on the keys files.
- Query (host):

```
for f in /root/.ssh/authorized_keys /home/*/.ssh/authorized_keys; do echo "== $f"; cat "$f" 2>/dev/null; done
```

- Corroborate: the key's modification time against an auth.log session for that account.

---

5. Evaluate the result

Three verdicts, and only three.

- **Present.** The query returned records consistent with the technique and a second source agrees. This is a finding. Record both sources.
- **Absent.** The query returned nothing, the source was confirmed collected for the window, and the baseline for that source is understood. This is a defensible negative.
- **Not collectable.** The mapped source was not collected, not forwarded, or not retained for the window. This is a collection gap. It is neither a hit nor a clear.

A query that returns records is not automatically a hit. Apply the baseline first — known-good automation, management hosts, and sanctioned tooling are cleared against the baseline register before a record is escalated. Only records that survive the baseline are evaluated against the technique.

Record which verdict applies and the evidence for it. "Absent" and "not collectable" look identical in an empty result set and must never be reported as the same thing.

---

6. Indicator crosswalk

Atomic indicators resolve to data sources the same way techniques do.

Indicator	Collected in	Query field
File hash	EDR, Sysmon 1/6/7, Amcache, Prefetch	`process.hash.sha256` / `file.hash.sha256` / Amcache first-seen
IP address	Zeek `conn`, firewall, Sysmon 3, proxy	`destination.ip` / `source.ip`
Domain	Zeek `dns` / `http` / `ssl`, DNS client, proxy	`dns.question.name` / `url.domain` / `tls.server.name`
Filename / path	`$MFT`, Sysmon 11, EDR	`file.path` / `file.name`
JA3 / JA4	Zeek `ssl`, Suricata	`tls.client.ja3` / `tls.client.ja4`
Email sender / subject	mail gateway	gateway index

An indicator hit is still one source. A hash seen in Amcache is corroborated by the execution artifact and the network the process generated; a domain in the DNS log is corroborated by the host and process that resolved it. Resolve a hit to the host, then run the host through the technique crosswalk to establish what the indicator did, not only that it was present.

---

7. Pivot — one technique implies its neighbors

A confirmed technique is a position in an intrusion, not the whole of it. Each tactic has predecessors and successors. A confirmed technique queues the adjacent techniques for the same host and window.

```
Initial Access -> Execution -> Persistence -> Priv Esc -> Defense Evasion
       -> Credential Access -> Discovery -> Lateral Movement -> C2 -> Collection -> Exfil -> Impact
```

Worked expansions:

- `T1059.001` PowerShell confirmed → query the parent (`T1566`, `T1203`), the network child (`T1071`, `T1105`), and any persistence it wrote (`T1053`, `T1547`).
- `T1003.001` LSASS access confirmed → identify the exposed account, then query that account for `T1021.001/002` to the hosts it reached.
- `T1021.002` SMB lateral confirmed → at each destination, query `T1569.002` / `T1053.005` for the execution method and restart the loop on that host.
- `T1071.001` beacon confirmed → resolve the owning process, then query its persistence and the same destination across every host.

Pivoting ends when the queued techniques return absent or not-collectable and the timeline is continuous from first activity to last.

---

8. Confidence and reporting

Two independent judgments, reported separately.

- **Evidentiary confirmation** — present, absent, or not collectable, from Section 5. This is about the data.
- **Analytic confidence** — how strongly the evidence supports the assessment. Use the standard gradations: *almost certainly, highly likely, likely, roughly even chance, unlikely, very unlikely.* State the basis — source reliability, corroboration count, collection completeness — and state what would raise or lower it.

A technique can be confirmed present with low analytic confidence in attribution, and that distinction is preserved in the report. Evidence of PsExec execution is high-confidence as a technique and says nothing by itself about which actor ran it.

Finding record:

```
INTEL IN:    source, date, input as received
TECHNIQUE:   T-code and name; tool/procedure if resolved
SCOPE:       hosts/segments queried, window (UTC)
QUERY:       the exact query run, per source
RESULT:      present / absent / not-collectable, with counts
EVIDENCE:    primary source + independent corroborating source, each hashed where applicable
CONFIDENCE:  analytic gradation + basis + what would change it
PIVOT:       techniques queued from this finding
GAPS:        sources that were not collectable for this technique
```

---

9. Collection gaps

A technique whose data source was not collected cannot be confirmed or cleared. Do not report it as absent. Record it as a named gap: the technique, the data component it needs, and the collection that would satisfy it. The gap register feeds back to the collection plan as a requirement — this is the threat-informed input that raises collection above the platform default for the next iteration.

```
GAP:  <T-code> requires <data component> ; not collected because <no sensor / not forwarded / retention> ;
      satisfied by <specific collection> ; priority <by terrain x threat>
```

Reduced visibility is a finding in its own right. An adversary who cleared a log or operated where no sensor sees is still placed by the absence, and the gap is reported, not hidden by an empty query.

> Defensive and authorized. Applied to collected data from provided systems under rules of engagement.

Detection & Filter Pack — Suricata / Zeek / Elastic Defend / KQL

Purpose. The concrete queries, rules, and filters behind the dashboards — the detect stage, plus the filter emission that the baseline doc hands off to. One reference so the rule text lives in one place instead of scattered across four tools:

```
  [ COLLECT ]  ->  [ BASELINE ]  ->  [ VISUALIZE ]  ->  [ DETECT ]
                       |
                       +---- emits filters applied first ----+
```

---

1. Organizing idea: layers, and baseline filters first

Detection is layered by vantage point, and the layers cover each other's blind spots (same logic as the collection plan):

- **Network layer** — Suricata (signatures, protocol anomalies) + Zeek (behavior, metadata, logs). Breadth, no agent, blind on encryption payload.
- **Host layer** — Elastic Defend (behavioral/memory, response) + KQL hunts over EVTX/Sysmon/auditd. Depth, per-host, sees what the wire can't.

Baseline filters run before detection, not after. A confirmed-normal register item is suppressed at the earliest point it can be (agent event filter, Suricata suppress, Zeek exclusion, KQL negative). You tune *out* the known-good so the rules fire on what's left. Every section ends with how that stack emits a baseline filter (Section 7 collects the mapping).

Order of operations per layer: **baseline filter → broad hunt query → specific rule → alert**.

---

2. KQL hunt library

Run in Kibana Discover / as saved searches / as the basis for detection rules. KQL unless marked ES|QL. These are hunts (wide, for a human) that graduate to rules (narrow, for the alert engine) once tuned.

Host hunts

```
# Office / email spawning a shell  (T1566 / T1059)
process.parent.name:("winword.exe" or "excel.exe" or "powerpnt.exe" or "outlook.exe")
  and process.name:("cmd.exe" or "powershell.exe" or "wscript.exe" or "cscript.exe" or "mshta.exe")

# Encoded / download-cradle PowerShell  (T1059.001 / T1105)
process.name:"powershell.exe" and process.command_line:
  (*-enc* or *-EncodedCommand* or *FromBase64String* or *IEX* or *Invoke-Expression*
   or *DownloadString* or *Net.WebClient* or *-nop* or *-w hidden*)

# LOLBins with suspicious args  (T1218)
process.name:("rundll32.exe" or "regsvr32.exe" or "mshta.exe" or "certutil.exe"
   or "bitsadmin.exe" or "msiexec.exe")
  and process.command_line:(*http* or *\\temp\\* or *-urlcache* or *javascript:* or *.dll,*)

# LSASS access (credential dumping)  (T1003.001)  — Sysmon 10 / Defend
event.code:"10" and winlog.event_data.TargetImage:*lsass.exe*
  and not winlog.event_data.SourceImage:("*\\MsMpEng.exe" or "*\\wininit.exe")   # baseline known

# New service / scheduled task persistence  (T1543.003 / T1053.005)
event.code:("7045" or "4697" or "4698" or "4702")

# Lateral movement via explicit creds  (T1021 / T1078)
event.code:"4648"      # then pivot: one user.name -> many host.name

# Failed-logon spray / bruteforce  (T1110)
event.code:"4625"      # aggregate by source.ip / user.name, threshold on count

# Linux: suspicious execution from web/temp dirs  (T1505.003 webshell / T1059.004)
host.os.type:"linux" and process.executable:("/tmp/*" or "/dev/shm/*" or "/var/www/*")
  and process.parent.name:("apache2" or "httpd" or "nginx" or "php-fpm")   # web proc spawning shell
```

Network hunts

```
# Beaconing candidates (fixed-interval small conns to one external dst)  (T1071)
#  -> use ES|QL for interval stats:
FROM logs-zeek.*
| WHERE network.direction == "outbound"
| STATS conns = COUNT(*), bytes = SUM(network.bytes), uniq_dur = COUNT_DISTINCT(zeek.conn.duration)
    BY source.ip, destination.ip, destination.port
| WHERE conns > 20 AND bytes < 500000     # many small, regular conns
| SORT conns DESC

# Rare external destinations (first-seen)
destination.ip:* and not destination.ip:(<internal ranges>) and not destination.ip:(<baseline externals>)

# DNS tunneling / DGA markers  (T1071.004 / T1568)
zeek.dns.query:* and (dns_qlen > 50 or dns_entropy > 4)         # runtime fields
zeek.dns.qtype_name:"TXT"                                        # aggregate volume by source.ip
dns.response_code:"NXDOMAIN"                                     # spike by source.ip = DGA

# Self-signed / odd TLS  (T1573 / C2 over TLS)
tls.server.x509.issuer.distinguished_name : tls.server.x509.subject.distinguished_name   # self-signed
  or tls.client.ja3:(<known-bad JA3>)
  or (tls.client.server_name:* and not destination.ip:<expected for that SNI>)           # SNI/IP mismatch

# Long-lived external sessions
zeek.conn.duration > 3600 and network.direction:"outbound"
```

---

3. Suricata

Role. Signature + protocol-anomaly engine on the wire. Strong at known-bad (ET/ETPRO rulesets, IOC sigs, exploit sigs, protocol violations). Weak at stateful/behavioral detection over time (beaconing interval analysis) — push that to Zeek/Kibana. Runs best inline with both directions of flow (watch asymmetric routing from the collection plan).

Operate the rulesets, don't hand-write everything. Enable Emerging Threats Open (or ETPRO), enable the categories that match your terrain, disable what you can't see. Tune aggressively — an untuned ruleset on a range range is a wall of false positives.

Rule anatomy:

```
action proto src_ip src_port -> dst_ip dst_port (msg:"..."; flow:...; content:"..."; \
   pcre:"..."; classtype:...; sid:1000001; rev:1; metadata:...;)
```

Custom examples (defensive, for terrain-specific detection):

```
# Outbound cleartext to a known-bad / unexpected external on a monitored egress
alert tcp $HOME_NET any -> $EXTERNAL_NET any (msg:"CUSTOM Egress to flagged host"; \
   flow:established,to_server; ip_dst <flagged-ip>; classtype:trojan-activity; sid:9000001; rev:1;)

# Self-signed TLS to external (metadata)
alert tls $HOME_NET any -> $EXTERNAL_NET any (msg:"CUSTOM Self-signed cert outbound"; \
   tls.cert_issuer; content:!"CN="; classtype:protocol-command-decode; sid:9000002; rev:1;)

# Suspicious DNS TXT volume (tunneling indicator — pair with Kibana aggregation)
alert dns $HOME_NET any -> any 53 (msg:"CUSTOM DNS TXT query"; dns.query; \
   dns.opcode:0; content:"|00 10 00 01|"; classtype:policy-violation; sid:9000003; rev:1;)

# SMB lateral indicator (admin share from a workstation subnet)
alert smb $WKSTN_NET any -> $SERVER_NET 445 (msg:"CUSTOM Wkstn->Server admin share"; \
   flow:to_server; classtype:policy-violation; sid:9000004; rev:1;)
```

range / baseline suppression — knock down the range-ism noise (Gate 1 of the baseline doc) in `threshold.config`:

```
# Suppress a sig entirely from the range NAT/jump source  (BL-003 [RANGE])
suppress gen_id 1, sig_id 2000419, track by_src, ip <range-nat-ip>

# Rate-limit a noisy-but-real sig to one alert per 5 min per host
threshold gen_id 1, sig_id 9000003, type threshold, track by_src, count 1, seconds 300

# Suppress traffic-generator synthetic-user hosts from a web-category sig  (BL-007 [RANGE])
suppress gen_id 1, sig_id 2013028, track by_src, ip <traffic-gen-host-range>
```

Every `suppress` carries its `BL-ID` in a comment — the suppression is traceable back to a confirmed register item, or it doesn't go in.

4. Zeek

Role. The network's system-of-record and behavioral engine. Where Suricata says "this packet matched," Zeek says "here is every connection, DNS query, TLS handshake, HTTP request, and file seen, as structured logs you can hunt and baseline." Best for the stateful/over-time detection Suricata is weak at, and the richest source for the Network dashboard.

Logs to mine (ship to Kibana via Filebeat zeek module):

Log	What it gives	Hunt use
`conn.log`	every flow: duration, bytes, state, history	beaconing, long conns, top talkers, exfil ratios
`dns.log`	queries, responses, rcodes, types	DGA, tunneling, NXDOMAIN spikes, TXT volume
`http.log`	method, host, URI, UA, status	webshell URIs, rare UAs, suspicious downloads
`ssl.log + x509.log`	SNI, versions, cert subject/issuer/validity, JA3/JA3S	self-signed, rare JA3, SNI/IP mismatch
`files.log`	files seen, MIME, hashes	malware delivery, carving (with file extraction)
`notice.log`	Zeek's own detections	pre-built behavioral notices
`weird.log`	protocol anomalies	tunneling, evasion, malformed traffic

JA3/JA4 — load the JA3 package (or built-in JA4 on current Zeek) to fingerprint TLS clients/servers; baseline the known-good JA3 set, alert/flag the rest.

Notice framework — turn a hunt into a standing detection. Example (long outbound connection):

```
# long-conns.zeek  — notice on outbound sessions over 1h
@load base/frameworks/notice
redef enum Notice::Type += { Long_Outbound_Conn };
event connection_state_remove(c: connection) {
    if ( c$duration > 1hr && Site::is_local_addr(c$id$orig_h)
         && ! Site::is_local_addr(c$id$resp_h) )
        NOTICE([$note=Long_Outbound_Conn, $conn=c,
                $msg=fmt("Long outbound conn %s -> %s (%s)",
                         c$id$orig_h, c$id$resp_h, c$duration)]);
}
```

Intel framework — feed IOCs (domains, IPs, hashes, JA3) from threat intel; Zeek matches them across all logs and writes `intel.log`. This is the clean place to do IOC matching from the collection plan's intel inputs.

Baseline exclusion — the Zeek way to filter confirmed-normal is to *exclude in the detection script*, not to drop the log (keep the record, suppress the alert):

```
# in a notice policy hook — suppress the ism, cite the register ID
hook Notice::policy(n: Notice::Info) {
    if ( n$note == Long_Outbound_Conn && n$conn$id$resp_h == <backup-server> )
        break;   # BL-014 [BOTH] confirmed: nightly backup, not a finding
}
```

5. Elastic Defend (endpoint)

Role. The host behavioral + response layer. Real-time process/file/network/registry/DLL events, in-memory and behavioral protections (malware, ransomware, memory-threat, credential-access), and the response actions (isolate host, kill process, remote collect). This is what the collection plan's EDR deploy tree puts on the high-value hosts.

Enable:

- **Detection rules** — turn on the Elastic prebuilt rule set, filtered to the platforms you've deployed on (Windows/Linux). Start with the "recommended" + the ATT&CK coverage matching your threat (from the collection plan's technique list). Don't enable all 1000+ cold on a range — you'll bury yourself.
- **Endpoint protections** — malware (ML), ransomware (behavioral), memory-threat, and malicious-behavior, set to *detect* first on a range/sensitive client, *prevent* once tuned and authorized. Prevent on a production client without authorization is how you cause an outage.
- **Response** — enable host isolation / process kill only where ROE permits; this is a live action on someone's box.

Baseline filtering — two mechanisms, know the difference:

Mechanism	What it does	Use for
Event filter	Stops matching events from being *written/ingested* at the agent	High-volume known-good telemetry you don't want at all (range orchestration, traffic-generator process noise). Cheapest filter, earliest point.

Mechanism	What it does	Use for
Rule exception	Stops a *specific rule* from alerting on a match, event still ingested	Known-good that trips one rule but you still want the raw event (admin tool that looks like a LOLBin).
Trusted application	Excludes a binary/path/signer from *protections*	A known tool the malware/behavior engine flags (a red-team or admin utility you've sanctioned).

Example event filter (drop traffic-generator synthetic-user browser noise — baseline Gate 1):

```
# Event filter  (BL-007 [RANGE] traffic-generator)
event.category : "process" and process.parent.name : "<traffic-gen>.exe"
```

Example rule exception (admin's legit certutil use trips the LOLBin rule):

```
# Exception on "Certutil URL download" rule  (BL-022 [CLIENT] confirmed admin host)
host.name : "ADM-TOOLS-01" and user.name : "svc-patch"
```

Rules carried through: event filter for volume you never want (earliest, cheapest); exception when you still want the event; trusted-app only for protection engines. Every one cites its `BL-ID`; `[RANGE]` filters come off at the client. Prefer the **event filter** for pure range noise (keeps the index clean), the **exception** when the raw event still has hunt value.

---

6. Mapping the layers to the threat (coverage check)

Tie the pack back to ATT&CK so you can prove coverage (same validation idea as the collection plan Section 10):

ATT&CK (example)	Network (Suricata/Zeek)	Host (Defend/KQL)
T1071 App-layer C2	Zeek conn beaconing, ssl JA3, Suricata C2 sigs	Defend network events by process
T1071.004 DNS C2	Zeek dns (TXT/NXDOMAIN/entropy), Suricata dns	—
T1059.001 PowerShell	—	KQL 4104 / encoded PS, Defend behavioral
T1003 Cred dumping	—	Defend memory-threat, KQL Sysmon 10 LSASS
T1021 Lateral movement	Zeek smb/conn, Suricata smb sig	KQL 4648/4624 type 3/10
T1486 Ransomware	Zeek files (mass rename weird)	Defend ransomware behavioral
T1048 Exfil	Zeek conn bytes-out ratio, long conns	Defend network by process

A technique with no row on either side is a **detection gap** — log it, same as the collection and baseline docs. Coverage proven across all four docs is the point of building them as one pipeline.

---

7. Filter emission — one table, all stacks

The hand-off from the Baseline Register. A `confirmed` register item becomes:

Register class	Stack	Filter form	Earliest point
host process / user pattern (range noise)	Elastic Defend	**event filter**	at agent (not ingested)
host process tripping one rule	Elastic Defend	**rule exception**	at rule (event kept)
sanctioned tool flagged by protection	Elastic Defend	**trusted application**	protection engine
network sig false-positive / ism	Suricata	`suppress` / `threshold` by sig+host	sensor
network behavior known-good	Zeek	`Notice::policy` hook `break`	notice engine (log kept)
IOC allowlist	Zeek	intel-framework exclusion	intel match
any Kibana-visible flow/host/dns/tls	Kibana	pinned `NOT (...)` dashboard filter	query time

Three rules govern all of them, inherited from the baseline doc:

1. **Only `confirmed` items filter.** Provisional predictions never suppress.
2. **Every filter cites its `BL-ID`.** Traceable to a justification or it doesn't ship.
3. **`[RANGE]` filters are dropped at the client.** The environment marker is the switch.

> **Defensive use.** Every rule, query, and filter here is for authorized defensive detection and hunt. Prevention/response actions (Suricata inline drop, Defend prevent/isolate) are live actions — enable them only within ROE and after tuning in detect mode. Baseline filters reduce noise, never coverage: keep each one traceable so detection is never silently blinded.

ICS/SCADA Defensive Overview

Orientation for a host/network analyst dropped onto an OT network. Defensive framing: what the systems are, how they talk, where they're weak, and what an attack looks like on the wire and on the host. One rule above all others: **on OT, passive first.** An active scan that is harmless on IT can hang a PLC or trip a safety function. Do nothing that writes, probes, or floods until the asset owner clears it.

---

1. The Systems

ICS (Industrial Control System) is the umbrella. **SCADA** (Supervisory Control and Data Acquisition) is the geographically distributed kind — pipelines, grid, water — where a central master polls remote sites. **DCS** (Distributed Control System) is the plant-local kind — a refinery or factory, control staying inside the facility. Same components, different spread.

Component	Role	Analyst note
PLC	Programmable Logic Controller — runs the physical process (valves, motors) on a scan loop	The target. Logic lives in its memory; "program download" changes the process
RTU	Remote Terminal Unit — field data collection at a remote SCADA site	Like a PLC, built for remote/telemetry links
IED	Intelligent Electronic Device — protective relay, breaker controller (power)	Speaks IEC 61850; trips breakers
HMI	Human-Machine Interface — operator's screen/controls	Usually Windows. Your most likely host-forensics target
EWS	Engineering Workstation — where logic is written and pushed to PLCs	Highest-value host. Owns the keys to the process
Historian	Time-series database of process values	Windows/DB. Bridges OT and IT; common pivot
SIS	Safety Instrumented System — independent safety shutdown	Hands-off. Target of TRITON. Interaction can endanger people

Purdue model — the reference segmentation. Know which level you're on; it sets what's normal.

```
L5  Enterprise (corporate IT, internet)
L4  Business / plant IT (email, ERP)
---- IT / OT boundary ----
L3.5 Industrial DMZ   <- historian replica, jump host, patch/AV live here
L3  Site operations   (historian, domain, EWS)
L2  Supervisory       (HMI, SCADA servers)
L1  Control           (PLC, RTU, IED, SIS)
L0  Process           (sensors, actuators — the physical world)
```

Traffic that crosses levels it shouldn't — L4 reaching straight to an L1 PLC — is the single most useful anomaly to look for.

---

2. The Protocols

OT protocols were built for reliable real-time telemetry on trusted serial loops, then lifted onto TCP/IP. Most carry **no authentication and no encryption** by design. On a flat or poorly segmented network that is the whole problem: anyone who can route a packet can command a controller.

Protocol	Port	Domain	Auth/crypto	Watch for
Modbus TCP	502/tcp	General / manufacturing	None	Write-single/multiple (FC 5,6,15,16) from an unexpected host; FC 8 diagnostics
DNP3	20000/tcp,udp	Electric / water SCADA	None (Secure Auth rarely on)	Operate/write, cold/warm restart, time-change from a non-master
EtherNet/IP + CIP	44818/tcp, 2222/udp	Manufacturing (Rockwell)	None	CIP program download, "forward open" to controllers, reset services
S7comm / S7+	102/tcp	Siemens	Weak/none (S7+ obfuscated)	PLC STOP, program upload/download, run-state change
OPC UA	4840/tcp	Modern integration	Supports it (often off)	Unencrypted sessions, anonymous login
OPC DA (DCOM)	135 + dynamic	Legacy integration	Weak	Wide dynamic RPC range = firewalling nightmare
IEC 60870-5-104	2404/tcp	Electric (Europe/Asia)	None	C_SC/C_DC control commands, interrogation from unknown source
IEC 61850 MMS	102/tcp	Substations	None	Control to IEDs
IEC 61850 GOOSE / SV	L2 multicast (no IP)	Substations, fast trip	None	Spoofed GOOSE with high stNum to force a trip; won't show in an IP capture

Protocol	Port	Domain	Auth/crypto	Watch for
BACnet/IP	47808/udp	Building automation (HVAC)	None	Write-property to controllers
PROFINET	L2 + 34962-34964	Siemens fieldbus	None	Layer-2; needs a SPAN on the right segment

Rule of thumb: these are **plaintext**. In a pcap you can usually read the command intent directly. That's bad for the defender's network and good for the defender's packet analysis.

---

3. Common Weaknesses

Ordered roughly by how often they're the finding. Most OT incidents are not zero-days; they're these.

1. **Flat networks / weak segmentation.** No real boundary between IT and OT, or between Purdue levels. One phished L4 workstation reaches L1. Look for routes and sessions that cross levels.
2. **Internet-exposed OT.** HMIs, PLCs, and VNC/RDP reachable from the internet (findable on Shodan/Censys). Check for public-facing 502, 44818, 20000, 47808 and remote-access ports.
3. **No authentication on control protocols.** By design (§2). Compensated only by segmentation — so #1 and #3 compound.
4. **Default / shared / hardcoded credentials.** Vendor defaults on HMIs, switches, and web UIs; shared operator logins with no attribution.
5. **Unpatched, end-of-life systems.** Windows 7/XP HMIs, decade-old firmware. Patching is hard because uptime is sacred — so legacy CVEs persist for years.
6. **Dual-homed hosts bridging IT and OT.** Historians and EWS with a NIC in each world — the designed-in pivot path. Enumerate multi-homed assets.
7. **Unmanaged remote access.** Vendor maintenance via TeamViewer/VNC/cellular modems, often outside the firewall and unlogged.
8. **Removable media & the EWS.** USB is how air-gapped plants get updates — and malware (Stuxnet). The EWS is the crown jewel: own it and you own the logic pushed to every PLC.
9. **No OT monitoring.** Many sites have zero packet visibility below L3. You often can't answer "is this normal?" because nobody ever baselined it. Baselining is itself a deliverable.

---

4. Identifying Attacks / Manipulation

OT favors the defender in one way: the process is **repetitive and predictable**. Normal traffic is the same masters polling the same slaves at the same intervals with the same function codes. Malice shows up as deviation from that rhythm.

On the wire

- **New talkers.** A host that never spoke a control protocol suddenly issuing Modbus writes or S7 commands. Baseline who-talks-to-whom; flag the new edge.
- **Write/control where you normally see reads.** Engineering functions (program download, write-multiple, operate, restart) outside a maintenance window, or from an HMI instead of the EWS.
- **State changes.** PLC STOP/RUN transitions, time-sync changes, firmware-update messages you didn't schedule.
- **Scan signatures.** Sequential unit-ID / register sweeps, connection attempts across the whole L1 range — recon on a network that should have fixed, stable flows.
- **GOOSE anomalies.** Sudden jump in stNum/sqNum, or two publishers for one dataset — a spoof forcing a protective trip. Needs an L2 capture on the station bus.
- **Timing.** Poll intervals that change, or a flood that delays real telemetry — availability attacks matter more than confidentiality in OT.

On the host (HMI / EWS / historian — usually Windows)

- Your §1 live-triage and §7–§8 disk/memory flows from the Execution Reference apply. Additions for OT:
  - **Engineering-software processes or project files changing** outside a maintenance window (TIA Portal, Studio 5000, etc.).
  - **New/unknown executables on the EWS** — the staging point for logic tampering.
  - **USB insertion events** on air-gapped hosts (EVTX, registry USBSTOR).
  - **Historian queries or exports** that don't match normal reporting — data theft or recon.
  - **Project-file hashes** against a known-good baseline to catch logic changes.

Frameworks to anchor findings

- **MITRE ATT&CK for ICS** — map each observable to a technique (e.g. *Modify Controller Tasking*, *Unauthorized Command Message*, *Loss of Availability*). It's the ICS-specific matrix, not Enterprise.
- **NIST SP 800-82 Rev 3** — the authority for OT security architecture and controls.
- **CISA ICS advisories** — current vendor/product vulns and active-threat alerts.

Named ICS malware — the historical pattern (study what each abused)

- **Stuxnet** — USB → EWS → Siemens S7 PLC logic tampering; hid from the HMI.
- **Industroyer / Industroyer2** — speaks grid protocols natively (IEC 101/104, 61850, OPC) to trip breakers.
- **TRITON / TRISIS** — targeted the Triconex **SIS**; the line that should never be crossed.
- **PIPEDREAM / INCONTROLLER** — modular toolkit for Modbus, CODESYS, OPC UA; the first "reusable" ICS attack framework.
- **FrostyGoop** — Modbus TCP to heating controllers; recent, simple, effective.

The lesson across all of them: the access path is ordinary IT (phishing, USB, remote access, a flat network), and the OT-specific payload only fires at the end. **Defending the IT→OT boundary and the EWS stops most of the kill chain before it reaches the process.**